新規プロジェクト作成モーダル UI仕様書

1. 概要

1.1 コンポーネントの目的

新規プロジェクト作成モーダルは、プロジェクト管理画面から新規プロジェクトを作成するためのモーダルダイアログです。プロジェクト名と概要を入力し、バックエンドAPIに送信してプロジェクトを作成します。

1.2 使用場面

- プロジェクト管理画面のツールバー → 「新規作成」ボタンクリック時に表示
- モーダル形式で画面中央に表示

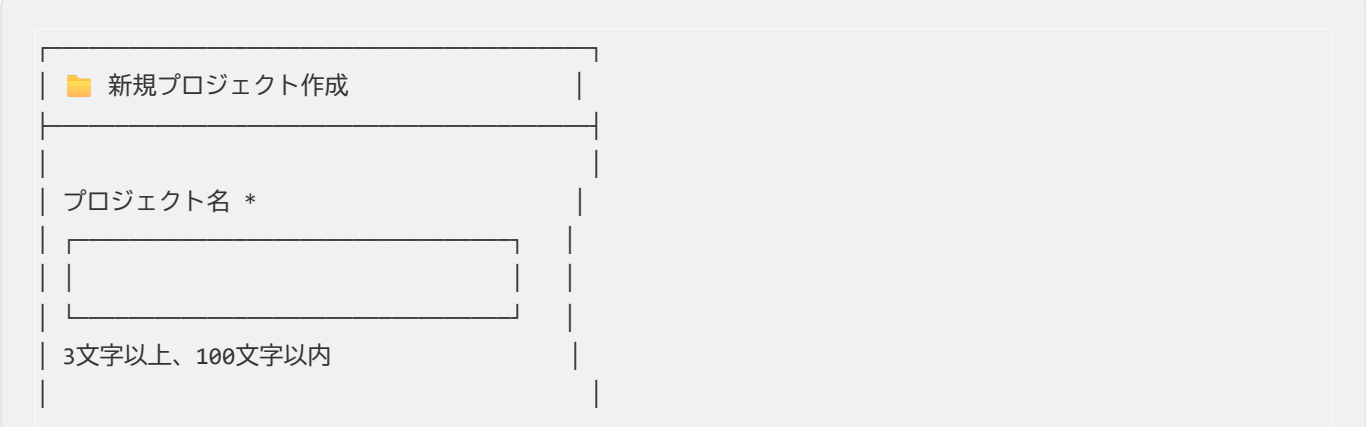
1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造



```
| 概要                                  |
| ┌──────────────────────────────┐   |
| │                              │   |
| │                              │   |
| │                              │   |
| └──────────────────────────────┘   |
| 500文字以内（任意）                   |
|                                      |
|              [キャンセル] [作成]      |
└──────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 モーダルヘッダー

- アイコン: FolderAddOutlined（プライマリカラー）
- タイトル: "新規プロジェクト作成"（多言語対応）

2.2.2 フォームフィールド

1. プロジェクト名（必須）

- タイプ: Input
- バリデーション:
  - 必須入力
  - 最小3文字
  - 最大100文字
- 文字数カウント表示
- オートフォーカス

2. 概要（任意）

- タイプ: TextArea
- 行数: 4行
- バリデーション:
  - 最大500文字

- 文字数カウント表示

2.2.3 フッターボタン

- **キャンセルボタン**: モーダルを閉じる、フォームをリセット
- **作成ボタン**: フォーム送信、ローディング状態表示

2.2.4 エラー表示

- Alert（error）: エラーメッセージ表示
- 閉じるボタン付き

---

3. 機能仕様

3.1 プロジェクト作成

3.1.1 処理フロー

```
const handleSubmit = async () => {
  try {
    setError(null);
    setSubmitting(true);

    // バリデーション
    const values = await form.validateFields();

    // プロジェクト名からフォルダ名を自動生成
    const folderName = values.projectName
      .replace(/\s+/g, '_')
      .replace(/[^a-zA-Z0-9_-]/g, '_');

    // プロジェクトデータの作成
    const projectData: CreateProjectData = {
      projectName: values.projectName,
      description: values.description || '',
      folderName: folderName,
      owner: currentUser.name,
      department: currentUser.department,
```

```
      organizationId: currentUser.organizationId,
    };

    // 親コンポーネントの送信ハンドラを呼び出し
    await onSubmit(projectData);

    // 成功時はフォームをリセット
    form.resetFields();
  } catch (err) {
    if (err instanceof Error) {
      setError(err.message);
    } else {
      console.error('Form validation failed:', err);
    }
  } finally {
    setSubmitting(false);
  }
};
```

3.1.2 フォルダ名の自動生成

```
const folderName = values.projectName
  .replace(/\s+/g, '_')      // スペースをアンダースコアに変換
  .replace(/[^a-zA-Z0-9_-]/g, '_');  // 使用不可文字をアンダースコアに変換
```

例:

- `"プロジェクト 001"` → `"_____001"`
- `"Project-ABC"` → `"Project-ABC"`
- `"Test Project (2026)"` → `"Test_Project__2026_"`

3.1.3 UI動作

1. 作成ボタンクリック
2. フォームバリデーション実行
3. バリデーション成功: プロジェクトデータを作成
4. 親コンポーネントの `onSubmit` コールバック呼び出し
5. 成功: モーダルを閉じる、フォームリセット

6. 失敗: エラーメッセージ表示

3.2 キャンセル

3.2.1 処理フロー

```
const handleCancel = () => {
  form.resetFields();
  setError(null);
  onCancel();
};
```

3.2.2 UI動作

- キャンセルボタンまたは×ボタンクリック
- フォームをリセット
- エラーメッセージをクリア
- モーダルを閉じる

3.3 バリデーション

3.3.1 プロジェクト名

```
rules={[
  {
    required: true,
    message: 'プロジェクト名を入力してください',
  },
  {
    min: 3,
    message: 'プロジェクト名は3文字以上で入力してください',
  },
  {
    max: 100,
    message: 'プロジェクト名は100文字以内で入力してください',
```

```
  },
]}
```

3.3.2 概要

```
rules={[
  {
    max: 500,
    message: '概要は500文字以内で入力してください',
  },
]}
```

3.4 多言語対応

```
const { t } = useLanguage();

// モーダルタイトル
t('projectManagement.create.title')  // "新規プロジェクト作成"

// フィールドラベル
t('projectManagement.create.projectName')  // "プロジェクト名"
t('projectManagement.create.description')  // "概要"

// プレースホルダー
t('projectManagement.create.projectNamePlaceholder')  // "プロジェクト名を入力"
t('projectManagement.create.descriptionPlaceholder')  // "プロジェクトの概要を入力（任意）"

// バリデーションメッセージ
t('projectManagement.create.projectNameRequired')  // "プロジェクト名を入力してください"
t('projectManagement.create.projectNameMinLength')  // "プロジェクト名は3文字以上で入力してください"
t('projectManagement.create.projectNameMaxLength')  // "プロジェクト名は100文字以内で入力してください"
t('projectManagement.create.descriptionMaxLength')  // "概要は500文字以内で入力してください"

// ボタン
t('common.create')  // "作成"
t('common.cancel')  // "キャンセル"
```

```
// エラー
t('projectManagement.create.error')  // "作成エラー"
```

---

4. データ仕様

4.1 Props定義

```
interface CreateProjectModalProps {
  /** モーダルの表示状態 */
  open: boolean;
  /** キャンセル時のコールバック */
  onCancel: () => void;
  /** 送信時のコールバック */
  onSubmit: (data: CreateProjectData) => Promise<void>;
  /** 現在のユーザー情報（デフォルト値あり） */
  currentUser?: {
    name: string;
    department: string;
    organizationId: string;
  };
}
```

Props説明:

Prop名	型	必須	説明
open	`boolean`	✅	モーダルの表示状態
onCancel	`() => void`	✅	キャンセル時のコールバック
onSubmit	`(data: CreateProjectData) => Promise<void>`	✅	送信時のコールバック
currentUser	`{ name, department, organizationId }`	-	現在のユーザー情報（デフォルト: テストデータ）

4.2 State定義

```
const [form] = Form.useForm();
const [submitting, setSubmitting] = useState(false);
const [error, setError] = useState<string | null>(null);
```

State説明:

State名	型	初期値	説明
form	`FormInstance`	-	Ant Design Form インスタンス
submitting	`boolean`	`false`	送信中かどうか
error	`string | null`	`null`	エラーメッセージ

4.3 CreateProjectData型定義

```
export interface CreateProjectData {
  projectName: string;      // プロジェクト名
  description: string;      // 概要
  folderName: string;       // フォルダ名（自動生成）
  owner: string;            // 所有者名
  department: string;       // 部門名
  organizationId: string;   // 組織ID
}
```

---

5. API連携

このコンポーネントは直接APIを呼び出しません。親コンポーネント（ProjectManagement）の `onSubmit` コールバックを通じてAPI呼び出しを委譲します。

親コンポーネントでの実装:

```
const handleCreateProject = async (data: CreateProjectData) => {
  const requestData: CreateProjectRequest = {
    projectName: data.projectName,
    description: data.description || undefined,
  };
```

```
  await createProjectApi(requestData);
  await refetchProjects();
  message.success('プロジェクトが正常に作成されました');
};
```

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
バリデーションエラー	フィールド下にエラーメッセージ表示
API呼び出しエラー	Alertコンポーネントでエラーメッセージ表示
ネットワークエラー	エラーメッセージ表示 + コンソールログ出力

6.2 エラー処理コード

```
try {
  setError(null);
  setSubmitting(true);
  const values = await form.validateFields();
  // ... プロジェクト作成処理
  await onSubmit(projectData);
  form.resetFields();
} catch (err) {
  if (err instanceof Error) {
    setError(err.message);
  } else {
    console.error('Form validation failed:', err);
  }
} finally {
  setSubmitting(false);
}
```

---

7. 実装ファイル

7.1 メインファイル

- **パス**: `frontend/src/components/project/CreateProjectModal.tsx`
- **行数**: 約170行
- **エクスポート**: `CreateProjectModal`（default export）、`CreateProjectData`（named export）

7.2 関連ファイル

- `frontend/src/contexts/LanguageContext.tsx` - 多言語対応

7.3 依存ライブラリ

- React 18.2.0
- Ant Design v6 (Modal, Form, Input, Space, Typography, Alert, theme)
- @ant-design/icons (FolderAddOutlined)

---

8. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を再作成	Cline

---

9. 補足事項

9.1 モーダル設定

```
<Modal
  title={<Space><FolderAddOutlined /><Text strong>新規プロジェクト作成</Text></Space>}
  open={open}
  onOk={handleSubmit}
  onCancel={handleCancel}
```

```
  confirmLoading={submitting}
  okText="作成"
  cancelText="キャンセル"
  width={600}
  destroyOnHidden  // モーダルを閉じた時にDOMから削除
/>
```

9.2 フォームレイアウト

```
<Form
  form={form}
  layout="vertical"           // 縦レイアウト
  requiredMark="optional"     // 必須マークをオプション扱い
>
```

9.3 デフォルトユーザー情報

```
currentUser = {
  name: '田中 太郎',
  department: '材料開発部',
  organizationId: 'org_maiml_lab_001',
}
```

- 開発・テスト用のデフォルト値
- 本番環境では親コンポーネントから渡される実際のユーザー情報を使用

9.4 フォルダ名生成の注意点

- 日本語はすべてアンダースコアに変換される
- 英数字とハイフン、アンダースコアのみ許可
- S3キーとして使用可能な文字に制限

9.5 送信ボタンの状態

- **通常**: "作成"
- **送信中**: スピナー表示 + "作成"（グレーアウト）
- **送信完了**: モーダルを閉じる

9.6 バリデーションタイミング

- フィールドのblur時
- フォーム送信時（validateFields）
- リアルタイム文字数カウント